This is what happens if we're running on a 64-bit platform and exceed the maximum integer size:

```
$largeNum = PHP_INT_MAX;

var_dump($largeNum);
var_dump(++$largeNum);

Output:
int(9223372036854775807)
float(9.223372036854776E+18)
```

PHP has converted the number to a `float` type because it isn't possible to represent *9223372036854775808* as an 8-byte integer. This allows us to count beyond the `PHP_INT_MAX` limit, but at the cost of precision. Notice that we have lost some significant digits after the conversion. `9.223372036854776E+18` in standard index notation is $9.223372036854776 \times 10^{18}$, which is *9223372036854776000*.

Without this change in data type, however, we would have a problem. When computers run out of bits to store ever larger integers, it triggers something called *integer overflow*. Think of it like a mechanical odometer on a car (if you're old enough to remember such things). As each number on the dial reaches nine, it spins back to the zero position and increments the next dial along. Eventually you run out of dials, and the entire odometer resets to zero. Some very old cars only stored five digits on theirs; when the mileage went to the next dial position from 99,999, it reset to 00,000. Something similar is happening in your computer's memory.

The 64-bit binary representation of `PHP_INT_MAX` is *01111111 11111111 11111111 1111111 11111111 11111111 11111111 11111111*. Remember, it is a signed integer, so the first bit is used for indicating positive or negative; in this case, it's set to zero. With signed integers, the leftmost bit is always negative, and its magnitude is equal to the largest number that can be represented with all the remaining bits set to "1". Therefore, for a 64-bit integer, the leftmost bit is equal to -2^{63}, or -9,223,372,036,854,775,808.

If we count one more than *01111111 11111111 11111111 1111111 11111111 11111111 11111111 11111111* we reach *10000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000*, which in 64-bit-signed-integer-ese is *-9,223,372,036,854,775,808 - 0* (the value of the left-most bit minus the values of the remaining 63 bits). Let's convert a few numbers beyond *PHP_INT_MIN* to binary to demonstrate how this wonky machine counting is working: